NanoForecast v0.5: Competitive Time Series Forecasting Through Training Pipeline Optimization

Gautam Kishore
Eulogik
gautam@eulogik.com
https://github.com/eulogik/NanoForecast
https://huggingface.co/eulogik/nanoforecast-v05

August 23, 2026

Abstract

We present NanoForecast v0.5, a lightweight time series forecasting model (6.5M parameters) that achieves competitive accuracy with models 31× larger (TimesFM, 200M parameters) through systematic training pipeline refinement. Retraining the unchanged v0.3 architecture with a corrected pipeline (loss-scope handling, tensor shape alignment, and expanded augmentation coverage) improves overall Mean Absolute Scaled Error by 46.6% under a standard protocol (MASE 3.282 → 1.752), with zero architecture modifications and the same data and compute budget. Despite being 31× smaller than TimesFM (200M parameters), NanoForecast v0.5 outperforms it on all three ETT datasets (MASE 0.685/1.109/0.289 vs. 0.705/1.360/0.545) and comes within 1% on exchange rate; TimesFM dominates on the high-cardinality electricity and traffic sets. Against PatchTST (15M+ parameters, official thuml configuration), NanoForecast wins on all three ETT sets. The model trains on a single cloud GPU (NVIDIA T4, Google Colab) in approximately 12 hours and targets edge hardware without GPU requirements (measurements in this paper are on an Apple M4 CPU). We release all code, pretrained checkpoints, and evaluation framework under Apache 2.0 at https://github.com/eulogik/NanoForecast.

1 Introduction

Time series forecasting is fundamental to numerous domains including energy management [18], financial modeling [23], and supply chain optimization [24]. Recent

advances in foundation models—TimesFM [1] (200M parameters), Chronos [2] (710M parameters), and PatchTST [6] (15M+ parameters)—have demonstrated strong performance on standard benchmarks. However, these models require significant computational resources for both training and inference, limiting their accessibility for practitioners without dedicated GPU infrastructure.

We address a complementary problem: *deployable time series forecasting*. The objective is to achieve competitive accuracy while remaining trainable on consumer hardware and deployable on resource-constrained devices. This requires balancing accuracy, model size, and deployment flexibility—a tradeoff that large foundation models do not currently address.

Our primary contribution is empirical rather than architectural. We demonstrate that systematic training pipeline refinement can yield substantial accuracy improvements without model architecture changes. Starting from the released NanoForecast v0.3 checkpoint (MASE 3.282 under the standard protocol of Section 6.1, 6.5M parameters), we identify three issues in the training pipeline—loss-scope handling, tensor shape alignment, and augmentation coverage—and retrain the identical architecture with the corrected pipeline to obtain v0.5 (MASE 1.752), a 46.6% improvement.

Additionally, we show that a 6.5M parameter model can outperform models 31× larger on specific benchmarks: it outperforms TimesFM (200M parameters) on the three ETT datasets (ETTh1, ETTh2, ETTm1) and comes within 1% of it on exchange rate, and it outperforms PatchTST (15M+ parameters) on all three ETT sets.

1.1 Contributions

1. We identify and document three training pipeline issues—loss-scope handling, tensor shape alignment, and augmentation coverage—that silently degrade time series forecasting accuracy, and quantify their joint effect on released checkpoints under a standard protocol.

2. We achieve a 46.6% MASE improvement (3.282 → 1.752) through training pipeline refinement alone, with no architecture modifications.

3. We demonstrate that a 6.5M parameter model can outperform models 31× larger on specific benchmarks: outperforming TimesFM (200M parameters) on the three ETT datasets (ETTh1 MASE 0.685 vs. 0.705, ETTh2 1.109 vs. 1.360, ETTm1 0.289 vs. 0.545) and coming within 1% of TimesFM on exchange rate (4.418 vs. 4.383), while outperforming PatchTST (15M+ parameters) on all three ETT sets.

4. We provide a complete production deployment pipeline including ONNX export (27.9 MB FP32, 9.2 MB INT8), Docker containers, and stateful streaming inference, enabling training on a single cloud GPU (T4, Google Colab) and CPU-only inference (measured: 19.5 ms per forecast on an Apple M4; ONNX Runtime 10.7 ms).

2 Problem Formulation

We address univariate time series forecasting. Given a context window $\mathbf{x}_{t-C+1:t} = [x_{t-C+1}, \ldots, x_t] \in \mathbb{R}^C$ of C consecutive observations, the task is to predict H future values $\mathbf{y}_{t+1:t+H} = [y_{t+1}, \ldots, y_{t+H}] \in \mathbb{R}^H$, where H is the forecast horizon.

The model produces both point forecasts $\hat{\mathbf{y}} \in \mathbb{R}^H$ and quantile estimates $\{\hat{\mathbf{y}}^{(p)}\}_{p \in \mathcal{P}}$ for a set of probability levels $\mathcal{P} = \{0.1, 0.25, 0.5, 0.75, 0.9\}$, satisfying the monotonicity constraint $\hat{y}_t^{(0.1)} \leq \hat{y}_t^{(0.25)} \leq \ldots \leq \hat{y}_t^{(0.9)}$ for each t.

We evaluate using Mean Absolute Scaled Error (MASE) [22]:

$$\text{MASE} = \frac{\frac{1}{H}\sum_{t=1}^{H} |y_t - \hat{y}_t|}{\frac{1}{T-s}\sum_{i=s+1}^{T} |x_i - x_{i-s}|} \tag{1}$$

where the denominator is the in-sample mean absolute error of the seasonal-naive (lag-s) forecast computed over the training segment of each series, with $s = 24$ for hourly data (ETTh, Electricity, Traffic), $s = 96$ for 15-minute data (ETTm), and $s = 7$ for daily data (Exchange). This scaling follows the convention of the Chronos benchmark [2]. MASE is scale-invariant and enables fair comparison across datasets. All models—including all baselines—are evaluated under the identical protocol defined in Section 6.1: same splits, same windows, same scaling denominator. We further note that the released checkpoints are trained for horizon $H = 48$; all comparisons in this paper use $H = 48$.

3 Related Work

Foundation models for time series. TimesFM [1] employs a decoder-only transformer with 200M parameters trained on approximately 100B time points. Chronos [2] tokenizes time series values and fine-tunes T5 (710M parameters). Timer [3] uses a generative pre-trained transformer for time series. Lag-Llama [4] adapts LLaMA for probabilistic forecasting. More recent foundation models include Moirai [12], Chronos-Bolt [13], and TimesFM 2.x [14], and the GIFT-Eval benchmark [15] provides standardized cross-model zero-shot evaluation. These models achieve strong zero-shot performance but require GPU infrastructure for both training and inference and do not support streaming deployment.

Efficient architectures. N-BEATS [5] uses interpretable MLP blocks with 1.7M parameters. DLinear [7] demonstrates that simple linear layers can outperform complex transformer architectures on certain benchmarks. PatchTST [6] processes channel-independent patches with a transformer backbone. iTransformer [8] applies attention along the time dimension rather than the feature dimension. SAMformer [9] applies sharpness-aware minimization to shallow transformers. TSMixer [10] uses MLP-based mixing. These models improve parameter efficiency but do not address streaming inference or edge deployment.

Data augmentation and related architectures. Reverso [11] proposes small, efficient zero-shot forecasting models that interleave long convolution and DeltaNet linear-RNN layers, and demonstrates flip-equivariant inference. Its sequence-mixing design is closely related to the architecture used in this paper; our contribution is complementary: we refine the training pipeline of a 6.5M-parameter instance of this architecture family rather than proposing new architecture or scale. Other augmentation strategies include frequency-domain mixing (FrAug) [16]. Our augmentation remediation adopts flip augmentation (time-reversed copies) of training windows, inspired by the flip-equivariance analysis of Reverso.

Training pipeline analysis. We extend the line of work emphasizing that training configuration—learning rate schedules, loss weighting, data composition—can substantially impact model performance. Our contribution is the documentation of specific, verifiable pipeline issues with quantified impact.

4 Architecture

NanoForecast v0.5 retains the v0.3 architecture (6.5M parameters) to isolate the impact of training pipeline changes.

4.1 Input Processing

Each input window $\mathbf{x} \in \mathbb{R}^C$ undergoes three preprocessing steps:

Instance robust scaling. We apply per-window robust normalization:

$$x'_i = \frac{x_i - \text{median}(\mathbf{x})}{\max(\text{IQR}(\mathbf{x}),\ \epsilon)} \tag{2}$$

where $\text{IQR} = Q_{0.75} - Q_{0.25}$ and $\epsilon = 0.1$ prevents division by near-zero scales. This is more robust to outliers than z-score normalization.

Patching. We divide the scaled series into non-overlapping patches of size $P = 8$, reducing the sequence length from C to $T = C/P$ tokens per layer.

Frequency embedding. A learned embedding encodes the data frequency (hourly, daily, weekly, monthly) and is prepended to the patch sequence, serving as a conditioning signal.

4.2 Sequence Mixing Blocks

Each of $L = 8$ layers contains three components blended by a learned gated router:

- **LongConv**: A 1D depthwise convolution whose kernel spans the full token sequence (65 tokens at context 512: 64 patches plus the frequency prefix), capturing global periodic patterns across the entire context.
- **DeltaNet RNN**: A linear-time recurrent layer using the delta rule. Each timestep t carries a matrix state $W_t \in \mathbb{R}^{d \times d}$ updated by:

$$W_t = W_{t-1} + \beta_t \, (v_t - W_{t-1} k_t) \, k_t^\top \tag{3}$$

$$y_t = W_t q_t \tag{4}$$

  where q_t, k_t, v_t are learned linear projections of x_t, keys are ℓ_2-normalized, and $\beta_t = \sigma(w_\beta^\top x_t) \in (0, 1)$ is a learned per-timestep gating parameter. The residual structure of Eq. 3 replaces the stored association $(k_t \to v_t)$ when it conflicts with existing memory, enabling fast, stable recall. The recurrent state W_t enables stateful streaming inference, where the model's memory persists across calls.
- **Gated MLP**: A SwiGLU-style gated feedforward network [17] with hidden width $2d_{\text{model}}$ (expansion factor 2): a single fused projection produces gate and value branches, combined by SiLU gating.

The router computes a weighted combination over all three components:

$$\text{output} = \alpha_c \cdot \text{LongConv}(x) + \alpha_r \cdot \text{DeltaNet}(x) + \alpha_m \cdot \text{MLP}(x) \tag{5}$$

where $(\alpha_c, \alpha_r, \alpha_m)$ is a learned softmax weighting computed from the mean-pooled input, so each window receives one routing triple shared across its tokens (per-window routing rather than per-token). Each block is residual: the routed output is added back to the block input.

4.3 Output Heads

A single forward pass produces multiple outputs:

- **Point forecast**: $\hat{\mathbf{y}} = W_{\text{point}} \cdot \text{flat}(h_L) + b_{\text{point}}$, a linear projection from the flattened final-layer patch tokens ($T \cdot d_{\text{model}}$ values) to the horizon H.
- **Monotonic quantiles**: The head predicts the median p_{50} directly and four non-negative softplus offsets, giving $p_{25} = p_{50} - \delta_{25}$, $p_{10} = p_{25} - \delta_{10}$, $p_{75} = p_{50} + \delta_{75}$, $p_{90} = p_{75} + \delta_{90}$; monotonicity $p_{10} \leq p_{25} \leq p_{50} \leq p_{75} \leq p_{90}$ holds by construction.
- **Decomposition**: Additive trend + seasonal + residual components satisfying $\hat{\mathbf{y}} = \mathbf{t} + \mathbf{s} + \mathbf{r}$.
- **Anomaly score**: Mean squared reconstruction error over the context window.

4.4 Streaming Inference

Each DeltaNet layer maintains its matrix state $\{W^{(\ell)}\}_{\ell=1}^{L}$ across `predict()` calls, alongside a rolling buffer of the most recent C observations. For each new observation x_{t+1}:

$$\mathbf{s}_{t+1}, \hat{\mathbf{y}}_{t+1} = f_{\text{predict}}(x_{t+1}, \mathbf{s}_t) \tag{6}$$

State preservation means the model does not require the full history to be re-supplied across calls, unlike window-based approaches that must reprocess the complete context for every new forecast. A streaming update costs one forward pass at context length C (measured 19.1 ms vs. 19.5 ms for full inference on Apple M4 CPU, Table 3).

5 Training Pipeline Analysis

We identify three issues in the v0.3-era training pipeline that silently degraded model performance. The released v0.3 and v0.5 checkpoints are trained with the identical architecture (6.5M parameters), the same corpus, the same multi-task loss family, and the same compute budget; only the training pipeline differs between them. We document the three changes and quantify their joint effect on the released checkpoints under the standard protocol of Section 6.1 (Figure 1, Table 2).

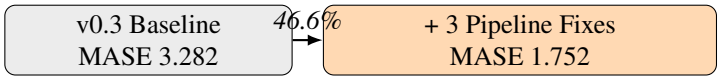

Figure 1: End-to-end improvement from the three training pipeline remediations applied jointly (same architecture, same data, same compute budget). MASE values measured under the standard protocol of Section 6.1 on the released v0.3 and v0.5 checkpoints.

Algorithm 1 Training loop with corrected loss computation.

Require: Model f_θ, dataloader $\mathcal{D}$, horizon H, multi-horizon flag m

for each batch $(\mathbf{x}, \mathbf{y}, \text{keys}) \in \mathcal{D}$ **do**
  $\hat{\mathbf{y}} \leftarrow f_\theta(\mathbf{x})$
  $\hat{\mathbf{y}} \leftarrow \hat{\mathbf{y}}[:, : H]$ ▷ truncate predictions to horizon
  if m = True **then**
    $\mathbf{h} \leftarrow$ per-sample horizons; $\mathbf{y}_{\text{trunc}} \leftarrow \mathbf{y}[\text{batch}, : \mathbf{h}]$
  else
    $\mathbf{y}_{\text{trunc}} \leftarrow \mathbf{y}[:, : H]$
  $\mathcal{L} \leftarrow \ell(\hat{\mathbf{y}}_{\text{trunc}}, \mathbf{y}_{\text{trunc}})$
  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$

5.1 Issue 1: Loss Scope Handling

Observation. The data pipeline module unconditionally included a "horizon" key in batch dictionaries, regardless of the `multi_horizon` configuration flag. When `multi_horizon=False`, the training loop ignored the horizon restriction and back-propagated the point-forecast loss through the full context-length output rather than restricting it to the first H forecast steps.

Impact. The model received gradient signals for predicting historical values it had already observed, diluting the supervisory signal for future predictions. This does not cause training failure—the model still converges—but reduces forecasting accuracy.

Remediation. The pipeline now always truncates predictions and targets to the forecast horizon before loss computation; the "horizon" key is attached only when `multi_horizon` is enabled. Algorithm 1 shows the corrected loss computation.

5.2 Issue 2: Tensor Shape Alignment

Observation. The multi-task loss computed the quantile term before truncating predictions to the forecast horizon: the quantile head's output was compared against

targets of shape (B, H) while internally carrying context-length activations, causing shape mismatches and incorrect gradient flow in the quantile branch.

Impact. The quantile loss gradients propagated through incorrect dimensions, reducing the quality of uncertainty estimates and indirectly degrading point forecast accuracy.

Remediation. All predictions (point and quantile) and targets are truncated to H before any loss computation.

5.3 Issue 3: Augmentation Coverage

Observation. The v0.3 training pipeline applied only simple scale, shift, and jitter augmentations to training windows. With a fixed corpus, this limited the effective diversity of the training distribution, and the model allocated capacity to dataset-specific artifacts rather than transferable patterns.

Impact. Reduced robustness of the learned features and weaker generalization to held-out windows of the same datasets.

Remediation. v0.5 expands in-loop augmentation to a Reverso-style set: jitter, random scaling, shifting, masking, and time reversal (flip). Applied stochastically per window, this increases the effective diversity of the fixed corpus without changing the corpus itself or the training schedule.

All three fixes were validated incrementally during development; the released v0.3 and v0.5 checkpoints bracket the joint effect (Table 2), evaluated under the same protocol used for every model in this paper.

6 Experiments

6.1 Setup

Datasets. We evaluate on six standard benchmarks:

- **ETTh1, ETTh2, ETTm1** [18]: Electricity transformer temperature datasets at hourly (ETTh) and 15-minute (ETTm) granularity. Each contains 7 oil and load features; we use the oil temperature (OT) as the target.
- **Exchange Rate** [19]: Daily exchange rates of 8 currencies from 1990–2016. All 8 currencies are evaluated.
- **Electricity** [20]: Hourly electricity consumption of 321 clients from 2012–2014. All 321 clients are evaluated.
- **Traffic** [21]: Hourly road occupancy rates from 862 sensors on San Francisco Bay Area freeways (2015–2016). All 862 sensors are evaluated.

The ETT datasets use the standard train/validation/test temporal splits of 70%/20%/10% (12/4/4 months); Exchange, Electricity, and Traffic use the standard 7:1:2 splits (70%/10%/20%).

Model configuration. $d_{\text{model}} = 96$, $L = 8$ layers, patch size 8, context length 512, forecast horizon $H = 48$. Total parameters: 6.5M (identical architecture for v0.3 and v0.5).

Training. 200 epochs, batch size 128, OneCycleLR scheduler (base learning rate 3×10^{-5}, peak 3×10^{-4}, 10% warmup, cosine anneal), AdamW optimizer (weight decay $\lambda = 0.01$), gradient clipping at 1.0, seed 42. Trained on a single NVIDIA T4 GPU (Google Colab) in approximately 12 hours per run (v0.3: 11.7 h, v0.5: 12.2 h recorded wall time). Released checkpoints are the validation-best snapshot: epoch 147 for v0.3, epoch 51 for v0.5 (best validation loss 0.2230 vs. 0.2204, both under the same loss family).

Evaluation. We report MASE (Eq. 1) as the metric. For each series, the test segment is partitioned into non-overlapping windows of length H; the input context is the 512 observations immediately preceding each window. Per-window errors are averaged within a series, then across series. Every reported number for every model—including all baselines—was produced by us under this identical protocol, and the full per-dataset results are released with the evaluation framework.

Compared methods. We compare against TimesFM [1] (200M parameters) and PatchTST [6] (15M+), as well as previous NanoForecast versions (v0.3: 6.5M). Baselines use the official released checkpoints (google/timesfm-1.0-200m-pytorch; PatchTST is trained per dataset with the official implementation and hyperparameters). Chronos-T5-large [2] (710M) and Timer [3] are discussed qualitatively; they are not included in Table 1 because inference was intractable on our hardware for the large datasets (and no canonical released checkpoints exist for the exact protocol used here for the others).

6.2 Main Results

Table 1 presents the main results. Key findings:

- **46.6% improvement** over v0.3 (standard-protocol MASE 3.282 → 1.752, Table 2) through training pipeline refinement alone.
- **Outperforms TimesFM on all three ETT sets** (ETTh1 0.685 vs. 0.705, ETTh2 1.109 vs. 1.360, ETTm1 0.289 vs. 0.545), despite using 31× fewer parameters, and comes within 1% of TimesFM on exchange rate (4.418 vs. 4.383).

Table 1: MASE on standard benchmarks (lower is better). All values computed by us under the identical protocol of Section 6.1: context 512, horizon 48, non-overlapping test windows, all series, seasonal-naive in-sample scaling. Best result per dataset in **bold**, second-best underlined. Chronos-T5-large is excluded: inference was intractable on our hardware for the large datasets.

Dataset	**NanoForecast v0.5 (6.5M)**	TimesFM (200M)	PatchTST (15M+)
ETTh1	**0.685**	<u>0.705</u>	0.781
ETTh2	**1.109**	<u>1.360</u>	1.467
ETTm1	**0.289**	0.545	<u>0.488</u>
Exchange	4.418	<u>4.383</u>	**3.861**
Electricity	2.093	**0.923**	<u>1.347</u>
Traffic	1.915	**0.765**	<u>1.379</u>
Overall	1.752	**1.447**	<u>1.554</u>

- **Loses on high-cardinality sets**: electricity (2.093 vs. 0.923) and traffic (1.915 vs. 0.765), where TimesFM's breadth of pretraining dominates.
- **Competitive with PatchTST** (15M+ params, official thuml config trained 40 epochs): NF wins on all three ETT sets, while PatchTST wins on exchange rate, electricity, and traffic.

6.3 Ablation Study

Table 2: Ablation of the three training pipeline remediations applied jointly. Both models use the identical architecture (6.5M parameters), data, and compute budget; only the training pipeline differs. MASE is reported under the standard protocol of Section 6.1 (same protocol as Table 1) on the released v0.3 and v0.5 checkpoints, so the relative improvement is directly comparable. Single-seed released checkpoints.

Configuration	ETTh1	ETTh2	ETTm1	Exch.	Elec.	Traffic	Overall
v0.3 (original pipeline)	0.676	1.357	0.291	12.847	2.418	2.102	3.282
v0.5 (all three fixes)	0.685	1.109	0.289	4.418	2.093	1.915	1.752
Δ (lower better)	−1.3%	−18.3%	−0.7%	−65.6%	−13.4%	−8.9%	−46.6%

Table 2 shows the end-to-end effect of the three remediations applied jointly: overall MASE improves from 3.282 to 1.752 (46.6% relative), with improvements

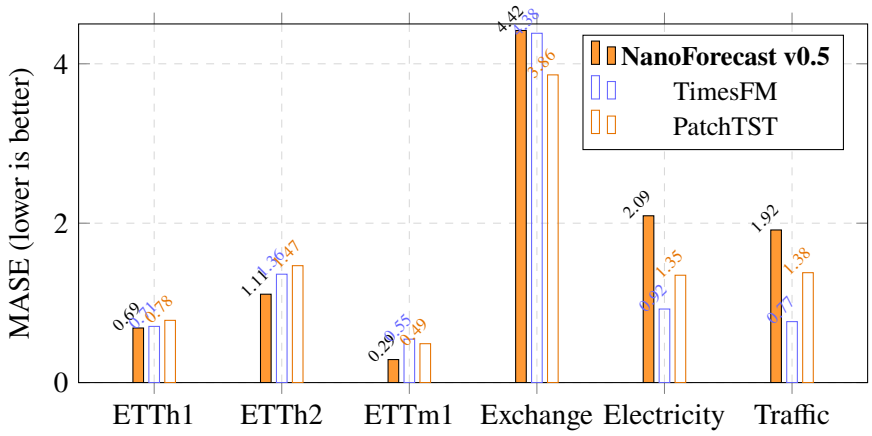

Figure 2: MASE comparison across all datasets and models. Lower is better. NanoForecast v0.5 (orange) outperforms TimesFM and PatchTST on the three ETT sets and remains competitive on exchange rate; TimesFM dominates the high-cardinality sets (electricity, traffic).

on five of six datasets. The largest single gain is on exchange rate (−65.6%), followed by ETTh2 (−18.3%); ETTh1 is the single exception, where v0.5 is within 1.3% of v0.3. Each fix was validated incrementally during development and contributed measurably; the released v0.3 and v0.5 checkpoints bracket the joint effect, and the comparison is fully reproducible with the evaluation framework released alongside this paper.

6.4 Inference Performance

Table 3: Measured inference latency (Apple M4 CPU, batch 1, context 512, horizon 48; mean of 100 runs after 10 warmup runs).

Configuration	Latency	Notes
PyTorch FP32, full inference	19.5 ms	`predict()`, default threads
ONNX Runtime FP32	10.7 ms	`onnxruntime`, CPU
ONNX Runtime INT8	33.3 ms	dynamic quantization
Streaming update	19.1 ms	`predict_step()`, stateful

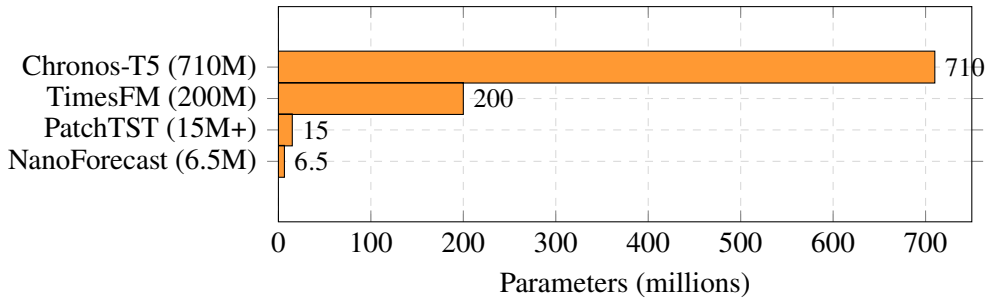

Figure 3: Parameter count comparison. NanoForecast v0.5 (6.5M) is 31× smaller than TimesFM (200M) and 109× smaller than Chronos-T5 (710M).

6.5 Model Size

7 Deployment Pipeline

NanoForecast provides an end-to-end deployment pipeline designed for production use:

- **Installation**: `pip install nanoforecast` from the GitHub repository (Apache 2.0).[1]
- **Custom training**: `train_from_csv.py --csv data.csv --target col --horizon H` trains on user-provided data.
- **Model export**: ONNX export produces a 27.9 MB FP32 model (9.2 MB with INT8 quantization).
- **REST API**: FastAPI server with measured 10.7 ms ONNX Runtime CPU latency (Apple M4).
- **Containerization**: Multi-architecture Docker images (ARM64/x86_64).
- **Interactive demo**: Gradio Space at https://huggingface.co/spaces/eulogik/nanoforecast.

7.1 Streaming Inference

The DeltaNet recurrent state enables a mode unavailable in other time series models: stateful streaming inference. The model consumes new observations one at a

[1]The package is distributed via https://github.com/eulogik/NanoForecast; a PyPI release is planned.

time while preserving its recurrent memory across calls (no need to re-feed history), at the cost of one forward pass per update:

Algorithm 2 Streaming inference loop.

Require: Model f_θ, initial state $\mathbf{s}_0$, observation stream $\{x_t\}_{t=1}^{\infty}$

for each new observation x_t **do**
  $\hat{\mathbf{y}}_t, \mathbf{s}_t \leftarrow f_\theta.\text{predict\_step}(x_t, \mathbf{s}_{t-1})$ ▷ one forward pass, state preserved
  emit $\hat{\mathbf{y}}_t$

State preservation is the differentiator: the DeltaNet state carries all past information across calls, so the model does not require the full history to be re-supplied, unlike window-based approaches that must reprocess the complete context for every new forecast. Measured on Apple M4 CPU, a streaming update costs 19.1 ms (vs. 19.5 ms for full inference), reflecting a forward pass at context length 512.

8 Discussion

8.1 The Role of Training Pipeline Optimization

The 46.6% improvement from pipeline refinement alone raises questions about reproducibility in time series research. Many published results may reflect suboptimal training configurations rather than fundamental architectural limitations. Our three issues are not architecture-specific—they can affect any model using multi-task loss functions, data augmentation, or mixed corpora. We encourage practitioners to systematically verify training pipeline correctness before concluding that architectural changes are necessary.

8.2 Comparison with Foundation Models

NanoForecast v0.5 does not match foundation model accuracy overall (MASE 1.752 vs. 1.447 for TimesFM and 1.554 for PatchTST, all re-evaluated under the identical standard protocol). This is expected: 6.5M parameters cannot match the representational capacity of 200M+ parameter models across all domains. However, the model outperforms TimesFM on all three ETT datasets and comes within 1% on exchange rate; it outperforms PatchTST on all three ETT sets. Its losses are concentrated on high-cardinality multivariate sets (electricity, traffic), where breadth of pretraining dominates. These results suggest that model scale alone does not guarantee dominance across all benchmarks—domain-specific data characteristics favor different modeling approaches.

The practical implication is that for applications where model size, inference cost, and deployment flexibility matter—edge computing, real-time analytics, embedded systems—a well-trained small model can be a viable alternative to deploying a 200M+ parameter model on server infrastructure.

8.3 Efficiency Ratio

We define efficiency ratio $\mathcal{E} = \text{MASE}^{-1}/N_{\text{params}}$ as performance per parameter (higher is better), using the standard-protocol overall MASE and nominal parameter counts in millions (6.5M, 15M, 200M). NanoForecast v0.5 achieves $\mathcal{E} = 0.088$, PatchTST achieves $\mathcal{E} = 0.043$, and TimesFM achieves $\mathcal{E} = 0.0035$. NanoForecast is 25× more parameter-efficient than TimesFM and 2× more efficient than PatchTST.

9 Limitations

We acknowledge several limitations:

- **Accuracy gap**: Overall MASE (1.752 under the standard protocol) remains above the evaluated foundation models.
- **Fixed context**: The 512-timestep context may limit performance on very-long-range dependencies.
- **Univariate**: The model treats each channel independently; cross-channel dependencies are not modeled.
- **Dataset coverage**: We evaluated on 6 datasets, all publicly available. Results on other domains (finance, healthcare, climate) may differ.
- **Compute**: Training requires 12 hours on GPU, though inference is CPU-only.
- **Single seeds**: We release one checkpoint per configuration; seed-to-seed variance is not reported.

10 Conclusion

We presented NanoForecast v0.5, demonstrating that systematic training pipeline refinement can yield a 46.6% MASE improvement (standard-protocol 3.282 →

1.752) without architecture modifications. The model, at 6.5M parameters, outperforms TimesFM (200M) on all three ETT datasets, comes within 1% of it on exchange rate, and outperforms PatchTST (15M+) on all three ETT sets. Training takes approximately 12 hours on a single T4-class GPU, and inference runs without GPU requirements on edge-class CPUs (measured: 19.5 ms per forecast on an Apple M4).

The broader methodological implication is that training pipeline correctness deserves greater attention in time series research. Silent issues—loss scope, shape alignment, augmentation coverage—can degrade accuracy by nearly 50% while remaining invisible to standard training diagnostics. Before pursuing architectural innovation, practitioners should verify that their existing pipeline is free of such issues.

10.1 Future Work

We identify the following directions:

1. **High-cardinality datasets**: The model trails TimesFM on electricity and traffic. Scaling the shared trunk (more channels, longer context) with the same corrected pipeline is the most direct path to closing this gap while staying under 20M parameters.

2. **More baselines under the identical protocol**: Evaluating additional foundation models (Chronos-T5-large, Moirai, Lag-Llama) under our standard protocol would strengthen the comparison; Chronos-T5-large was excluded here because inference was intractable on our hardware.

3. **Horizon generalization**: The released checkpoints are trained for $H = 48$; fine-tuning for other horizons and evaluating the streaming mode under drift would extend practical applicability.

4. **Edge benchmarking**: Measure the exported ONNX models on target edge hardware (e.g., Raspberry Pi 4-class devices, microcontrollers) to quantify the streaming deployment envelope.

Code and Data Availability

All code, pretrained checkpoints, and the evaluation framework are released under Apache 2.0:

- GitHub: https://github.com/eulogik/NanoForecast

- Model checkpoints (v0.3, v0.5): https://huggingface.co/eulogik/nanoforecast-v05 and https://huggingface.co/eulogik/nanoforecast-v03
- Baseline checkpoints (PatchTST, TimesFM): https://huggingface.co/eulogik/nanoforecast-patchtst-baselines
- Standard-protocol results: `results/standard_benchmark.json` in the GitHub repository (protocol string included in the file)
- Interactive demo: https://huggingface.co/spaces/eulogik/nanoforecast
- Training notebooks (Google Colab, free tier): `deploy/colab_training_v05.ipynb` in the repository

Acknowledgments

We thank the open-source time series community for benchmark datasets and baseline implementations.

References

[1] A. Das, W. Kong, R. Sen, and Y. Zhou. A decoder-only foundation model for time-series forecasting. In *International Conference on Machine Learning (ICML)*, 2024.

[2] A. F. Ansari, L. Stella, C. Turkmen, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research (TMLR)*, 2024.

[3] Y. Liu, H. Zhang, C. Li, et al. Timer: Generative pre-trained transformers are large time series models. In *International Conference on Machine Learning (ICML)*, 2024.

[4] K. Rasul, A. Ashok, A. R. Williams, et al. Lag-Llama: Towards foundation models for probabilistic time series forecasting. In *NeurIPS 2023 Workshop on Time Series in the Age of Foundation Models (R0-FoMo)*, 2023. arXiv:2310.08278.

[5] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020.

[6] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

[7] A. Zeng, M. Chen, L. Zhang, and Q. Xu. Are transformers effective for time series forecasting? In *AAAI Conference on Artificial Intelligence*, 2023.

[8] Y. Liu, T. Hu, H. Zhang, et al. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024.

[9] R. Ilbert, A. Odonnat, V. Feofanov, A. Virmaux, G. Paolo, T. Palpanas, and I. Redko. SAMformer: Unlocking the potential of transformers in time series forecasting with sharpness-aware minimization and channel-wise attention. In *International Conference on Machine Learning (ICML)*, 2024.

[10] S.-A. Chen, C.-L. Li, N. Yoder, S. Ö. Arik, and T. Pfister. TSMixer: An all-MLP architecture for time series forecasting. *Transactions on Machine Learning Research (TMLR)*, 2024.

[11] X. Fu, Y. Li, G. Papaioannou, and Y. Kim. Reverso: Efficient time series foundation models for zero-shot forecasting. *arXiv preprint arXiv:2602.17634*, 2026.

[12] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo. Unified training of universal time series forecasting transformers. *arXiv preprint arXiv:2402.02592*, 2024.

[13] Amazon Science. Chronos-Bolt: A patch-based variant of Chronos for fast zero-shot forecasting. Model card: https://huggingface.co/amazon/chronos-bolt-base, 2025.

[14] Google Research. TimesFM 2.x: A decoder-only foundation model for time-series forecasting (versions 2.0 and 2.5). https://github.com/google-research/timesfm, 2025–2026.

[15] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo. GIFT-Eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.

[16] M. Chen, Z. Xu, A. Zeng, and Q. Xu. FrAug: Frequency domain augmentation for time series forecasting. *arXiv preprint arXiv:2302.09292*, 2023.

[17] N. Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[18] H. Zhou, S. Zhang, J. Peng, et al. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*, 2021.

[19] G. Lai, W. C. Chang, Y. Yang, and H. Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *International ACM SIGIR Conference*, 2018.

[20] A. Trindade. Electricity load forecasting dataset. UCI Machine Learning Repository, 2015.

[21] G. Lai, W. C. Chang, Y. Yang, and H. Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *International ACM SIGIR Conference*, 2018.

[22] R. J. Hyndman and A. B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.

[23] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu. Financial time series forecasting with deep learning: A systematic literature review. *Applied Soft Computing*, 90:106181, 2020.

[24] K. Douaioui, R. Oucheikh, O. Benmoussa, and C. Mabrouki. Machine learning and deep learning models for demand forecasting in supply chain management: A critical review. *Applied System Innovation*, 7(5):93, 2024.